Velozity Business OS DELIVERY REPORT

Delivery report: phases 2–5

Status, evidence, risks and what remains before production.

7 September 2026	Phases 2–5	Aditya Malviya	Pre-production
Report date	Period covered	Prepared for	Status

Executive summary

All planned work for phases 2 through 5 is built, tested and merged. The platform now covers the full commercial lifecycle, gives clients their own portal, tracks renewals and profitability, and connects to Slack, WhatsApp, Gmail and Microsoft 365.

The platform is not yet in production. What remains is configuration and provider setup rather than development work.

4	307	0	96
PHASES DELIVERED	TESTS PASSING	KNOWN DEFECTS	API ENDPOINTS

What was delivered

PHASE	SCOPE	STATUS
Phase 2 Production readiness	Amazon SES email provider; browser test suite; preflight diagnostics; performance work; runtime version pinned	COMPLETE Code complete; two credentials outstanding
Phase 3 Client portal	Six-screen client application; deliverable approval; per-person permissions; secure document delivery; invite and revoke	COMPLETE
Phase 4 Revenue retention	Renewal engine with mandatory loss reasons; retention reporting; pipeline forecast; delivery profitability	COMPLETE
Phase 5 Integrations	No-code automation builder; Slack; WhatsApp; Gmail and Microsoft 365 mailbox sync	BUILT Three of four not verified live

Business impact

The client portal changes what clients see

Clients can now follow their own projects, read published reports, download documents, see outstanding invoices and formally accept or reject deliverables — without emailing anyone. Deliverable acceptance in particular moves a conversation that usually lives in email into a recorded decision attributed to a named person on a specific date.

Renewals become a process rather than a reminder

Previously the system sent a notification when a contract was expiring and recorded nothing about what happened next. Every expiring contract now opens a renewal with an owner and an outcome, and **a lost renewal cannot be recorded without a reason.** Those reasons are counted, which turns churn from something observed into something that can be acted upon.

Profitability is visible per engagement

Invoiced revenue against recorded delivery cost, with hours used against estimate, for every project. Restricted to those permitted to see cost.

Automations no longer need a developer

Business users can now build their own rules. The editor is generated from the engine's own definitions, so it cannot produce a rule the system would reject.

Verification

307 automated tests pass. They fall into two kinds, which answer different questions.

SUITE	TESTS	WHAT IT PROVES
Security	38	Every attack named in the specification is performed and refused
AI guardrails	34	The model cannot write; approval precedes execution; injection defences hold
End-to-end lifecycle	25	Lead through to a provisioned project, in one run
Client portal isolation	22	A client sees their own data and nothing else
Automation engine	20	Conditions, cooldown, loop protection, and that no send action exists
Slack & WhatsApp	19	Request shape, and configuration faults distinguished from transient ones
Contract rendering	13	Deterministic output; failure on a missing variable
Opportunity lifecycle	12	Stage guards and the transition channel
Permission catalogue	12	Code and database agree exactly on who may do what
Tenant isolation	11	One customer cannot read another's data
Renewals	11	The state machine, and that a loss must record a reason
Amazon SES	11	The message it builds, and how it classifies failure
Seed data	11	Demonstration data is coherent
Inbound email	10	Matching, deduplication, and that inbound can never become outbound
Automation builder	10	The editor cannot drift from what the engine accepts
Forecasting	8	Weighted pipeline; unconvertible currency excluded and counted
Assistant conversation	6	Multi-turn transcripts are always well formed
Navigation	5	Exactly one section is highlighted at a time
Browser suite	**29**	**Every page renders, loads and responds to a click in a real browser**

Tests run against genuine PostgreSQL, so the security policies under test are the ones that run in production. No database server or Docker is required.

> **Why the browser suite was added**
>
> During this work a stale build left every page in the application rendering perfectly and responding to nothing — every button was inert. All the logic tests passed throughout, because rendered markup is not evidence that a page works. The browser suite exists to close exactly that gap, and its central assertion was itself verified by disabling JavaScript and confirming the test fails. A test written for a silent failure is worthless until it has been seen to fail.

Defects found and fixed during this work

Four of these would have reached production and none were visible by inspection.

DEFECT	CONSEQUENCE HAD IT SHIPPED
Currency conversion called with the wrong arguments	Every forecasting and renewal figure would have failed at runtime. No previous code used this function, so nothing else would have caught it.
Renewal safety trigger misconfigured	Every attempt to update a renewal would have failed.
Sign-in routing regression	Users signing in would have been returned to the login form. Introduced during this work; caught by the browser suite.
Misleading AI error message	An unconfigured AI key was reported as a service outage, sending anyone diagnosing it to the wrong place entirely.
Two navigation items highlighted at once	Cosmetic.

Risks and limitations

> **Integrations have not been exercised against live services**
>
> Amazon SES, Slack, WhatsApp, Gmail and Microsoft 365 are implemented and tested against simulated responses. **No real message has been sent or received through any of them,** because each requires an account this project does not have — and Gmail and Microsoft additionally require registered applications and customer consent.
>
> The code that builds each request and interprets each failure is verified. The network round trip is not. Expect to spend time on each provider's approval and configuration process; that time is not development work but it is real.

> **Security credentials were mishandled during setup**
>
> Over the course of configuration, live credentials twice ended up in files intended for version control, and API keys were pasted into the wrong settings. Both files have been corrected and the ignore rules tightened so every credential file is excluded by default.
>
> **Recommended:** rotate the Supabase service key and the database password, both of which have now been exposed in plain text. This is precautionary, not evidence of a breach.

Smaller limitations, all documented

- The e-signature integration with Zoho has not been run against their live service. The manual signing workflow is complete and is what the automated tests use.
- Exchange rates must be loaded before any base-currency total will display. The system deliberately shows nothing rather than assuming a rate of one.
- Four tabs on the client detail screen are explanatory placeholders. The underlying data exists and is available through the API.
- The local development machine runs an older Node.js version than the platform now requires. This affects the local environment, not the deployed application.

Assessment

The platform is functionally complete against the specification and its safety properties are enforced where they cannot be bypassed — in the database rather than in application code. The test suite covers the behaviour that matters, including the failure modes that are invisible from the outside.

The honest position on integrations is that they are written and reasoned about carefully, but unproven against the real services. That is a normal state for integration work before accounts exist, and it should be planned for rather than discovered: budget time for provider approval processes, and expect one or two adjustments per provider once real responses arrive.

The recommendation is to proceed to a limited pilot — internal users first, then one client on the portal — before general release.